INSTITUTO POLITÉCNICO NACIONAL
ESCUELA SUPERIOR DE CÓMPUTO

Práctica 6: Transformers

NATURAL LANGUAGE PROCESSING

7CM2

INTEGRANTES:

VÁZQUEZ BLANCAS CESAR SAID

PROFESOR:

JORGE LUIS ROSAS TRIGUEROS

FECHA DE ENTREGA: 11 DE MAYO DE 2026

FECHA DE REALIZACION DE LA PRACTICA: 3 DE MAYO DE 2026

Marco Teórico

El Procesamiento de Lenguaje Natural (Natural Language Processing, NLP) es una rama de la inteligencia artificial que se encarga del desarrollo de técnicas y modelos capaces de analizar, interpretar y generar lenguaje humano de manera automática. Esta disciplina combina conocimientos de lingüística, aprendizaje automático y ciencias de la computación, permitiendo a las máquinas comprender grandes volúmenes de información textual.

Una de las tareas más importantes dentro del NLP es la generación de texto condicionada, la cual consiste en producir una salida en función de una instrucción o entrada dada. Este tipo de problema es ampliamente utilizado en asistentes virtuales, sistemas de recomendación y herramientas de automatización.

En el contexto de esta práctica, el problema abordado consiste en la generación de comandos de Linux a partir de instrucciones en lenguaje natural. Esta tarea puede considerarse como una forma de traducción semántica, donde el modelo debe interpretar la intención del usuario y transformarla en un comando válido del sistema operativo.

Tradicionalmente, este tipo de tareas se resolvía mediante reglas definidas manualmente o sistemas basados en plantillas. Sin embargo, estos enfoques presentan limitaciones importantes, ya que no son capaces de generalizar a nuevas formas de expresión del lenguaje.

Para superar estas limitaciones, se utilizan modelos basados en aprendizaje profundo, particularmente aquellos que emplean la arquitectura Transformer. Este tipo de modelos introduce el mecanismo de self-attention, el cual permite capturar relaciones contextuales entre palabras dentro de una secuencia, logrando una mejor comprensión del lenguaje.

Los modelos Transformer han demostrado un alto desempeño en tareas de NLP, especialmente en generación de texto. Entre estos modelos destacan los modelos de lenguaje autoregresivos, los cuales generan texto token por token basándose en el contexto previo.

En esta práctica se utilizó un modelo preentrenado de la familia LLaMA (Large Language Model Meta AI), el cual es un modelo de lenguaje basado en la arquitectura Transformer. Este modelo fue entrenado previamente con grandes volúmenes de texto, lo que le permite contar con un conocimiento general del lenguaje.

Para adaptar el modelo a la tarea específica de generación de comandos de Linux, se utilizó el proceso de fine-tuning, el cual consiste en ajustar un modelo preentrenado utilizando un conjunto de datos especializado. Sin embargo, el entrenamiento completo de estos modelos puede ser costoso en términos computacionales.

Por esta razón, se empleó la técnica LoRA (Low-Rank Adaptation), la cual permite realizar un ajuste eficiente modificando únicamente una pequeña parte de los parámetros del modelo. Esto reduce significativamente el consumo de memoria y tiempo de entrenamiento, manteniendo un buen desempeño en la tarea.

El conjunto de datos utilizado en esta práctica fue construido de manera personalizada y está compuesto por pares de entrada y salida. Cada entrada corresponde a una instrucción en lenguaje natural, mientras que la salida representa el comando de Linux correspondiente. Además, el dataset incluye múltiples formas de expresar una misma intención, lo que permite mejorar la capacidad de generalización del modelo.

Finalmente, el desempeño del modelo se evalúa mediante pruebas cualitativas, observando la capacidad del sistema para generar comandos correctos a partir de distintas formulaciones de entrada. Este enfoque permite verificar que el modelo no solo memoriza ejemplos, sino que también es capaz de adaptarse a nuevas instrucciones.

Material y Equipo

Para la realización de esta práctica se emplearon diversas herramientas de software orientadas al desarrollo y entrenamiento de modelos de lenguaje basados en la arquitectura Transformer.

En primer lugar, se utilizó el lenguaje de programación Python, debido a su amplia adopción en el ámbito del aprendizaje automático y su compatibilidad con múltiples bibliotecas especializadas en procesamiento de lenguaje natural.

Para la implementación del modelo se utilizó la biblioteca Transformers de Hugging Face, la cual permite trabajar con modelos de lenguaje preentrenados y facilita su adaptación a tareas específicas mediante técnicas de fine-tuning.

Asimismo, se empleó la biblioteca Datasets, también de Hugging Face, para la creación y manipulación del conjunto de datos. Esta herramienta permite trabajar de manera eficiente con grandes volúmenes de información estructurada en pares de entrada y salida.

Para realizar el ajuste fino del modelo se utilizó la biblioteca PEFT (Parameter Efficient Fine-Tuning), la cual permite aplicar técnicas como LoRA (Low-Rank Adaptation). Esta técnica reduce significativamente el número de parámetros que deben ser modificados durante el entrenamiento, haciendo el proceso más eficiente en términos de memoria y tiempo.

Adicionalmente, se utilizó la biblioteca TRL (Transformer Reinforcement Learning), específicamente la clase SFTTrainer, la cual facilita el entrenamiento supervisado de modelos de lenguaje en tareas de generación de texto.

El entrenamiento del modelo se llevó a cabo utilizando el framework PyTorch, el cual proporciona herramientas para la construcción y optimización de redes neuronales profundas.

También se emplearon bibliotecas auxiliares como:

- **Accelerate**, para optimizar el uso de recursos durante el entrenamiento
- **Bitsandbytes**, para mejorar la eficiencia en el manejo de memoria
- **JSON**, para la lectura del dataset
- **OS**, para la gestión de rutas y directorios

Modelo Utilizado

El modelo base utilizado en esta práctica fue:

- **unsloth/Llama-3.2-1B-Instruct**

Este modelo pertenece a la arquitectura **Transformer** y está diseñado para tareas de generación de texto condicionada. Su capacidad para comprender instrucciones en lenguaje natural lo hace adecuado para la generación de comandos de Linux.

Entorno de Ejecución

El desarrollo de la práctica se realizó en dos entornos principales:

- **Google Colab**, utilizado para el entrenamiento del modelo, aprovechando el acceso a recursos de GPU que permiten acelerar el proceso de aprendizaje
- **Entorno local**, utilizado para la ejecución de pruebas y validación del modelo entrenado

El modelo ajustado fue almacenado en Google Drive, lo que permitió su posterior descarga y uso en un entorno local sin necesidad de repetir el entrenamiento.

Desarrollo de la Practica

El desarrollo de esta práctica consistió en la construcción, entrenamiento y evaluación de un modelo de lenguaje capaz de generar comandos de Linux a partir de instrucciones en lenguaje natural. Este proceso se llevó a cabo en varias etapas, las cuales se describen a continuación.

1. Construcción del Dataset

Se diseñó un conjunto de datos personalizado compuesto por pares de entrada y salida. Cada entrada corresponde a una instrucción en lenguaje natural, mientras que la salida representa el comando de Linux equivalente.

El dataset incluye ejemplos variados que cubren diferentes categorías de comandos, tales como:

- Manejo de archivos (ls, cp, mv, rm)
- Visualización de contenido (cat, head, tail)
- Búsqueda de archivos (find)
- Procesamiento de texto (grep)
- Información del sistema (df, free, top)
- Red (ping, netstat, lsof)
- Permisos (chmod, chown)
- Compresión (tar, zip)
- Servicios (systemctl)

Además, se incluyeron múltiples formas de expresar una misma intención, lo que permite que el modelo aprenda a generalizar y no dependa de una única estructura de entrada.

Ejemplo de un par de datos:

```
{
    "input": "List all files in the current directory with detailed information",
    "output": "ls -la"
},
{
    "input": "Show system information including kernel version",
    "output": "uname -a"
},
{
    "input": "Search for a specific word in a file",
    "output": "grep 'word' file.txt"
},
{
    "input": "Display the content of a file",
    "output": "cat file.txt"
},
```

Figura 1: Primeros datos del dataset

2. Formato de Entrenamiento

Para facilitar el aprendizaje del modelo, los datos fueron transformados a un formato tipo conversación, en el cual se define un rol para el modelo y se establece claramente la estructura de entrada y salida:

```
def format_example(item):
    return {
        "text": (
            "You are a Linux command generator. "
            "Output only the command, no explanation.\n\n"
            f"User: {item['input']}\n"
            f"Assistant: {item['output']}"
        )
    }
```

Figura 2: Formato del Entrenamient

Este formato permite que el modelo genere únicamente el comando correspondiente, evitando respuestas con explicaciones adicionales.

3. Carga del Dataset

El dataset fue almacenado en formato JSON y cargado en el entorno de entrenamiento utilizando Python. Posteriormente, fue convertido a un formato compatible con la biblioteca **datasets**.

```python
with open(dataset_path, "r", encoding="utf-8") as f:
    data = json.load(f)

def format_example(item):
    return {
        "text": (
            "You are a Linux command generator. "
            "Output only the command, no explanation.\n\n"
            f"User: {item['input']}\n"
            f"Assistant: {item['output']}"
        )
    }

dataset = Dataset.from_list([format_example(x) for x in data])
```

Figura 3: Carga del dataset y conversión

4. Carga del Modelo Base

Se utilizó un modelo preentrenado de la familia LLaMA, el cual fue cargado mediante la biblioteca Transformers.

```python
tokenizer = AutoTokenizer.from_pretrained(base_model_id)

if tokenizer.pad_token is None:
    tokenizer.pad_token = tokenizer.eos_token

model = AutoModelForCausalLM.from_pretrained(
    base_model_id,
    dtype=torch.float16,
    device_map="auto"
)

model.config.use_cache = False
```

Figura 4: Carga del modelo

El modelo fue configurado para desactivar el uso de caché durante el entrenamiento:

5. Configuración de LoRA

Para realizar el ajuste fino de manera eficiente, se utilizó la técnica LoRA, configurando los siguientes parámetros:

```
lora_config = LoraConfig(
    r=4,
    lora_alpha=8,
    lora_dropout=0.05,
    bias="none",
    task_type="CAUSAL_LM",
    target_modules=[
        "q_proj",
        "v_proj",
    ]
)
```

Figura 5: Configuración del LoRA

Esta configuración permite modificar únicamente una parte del modelo, reduciendo el costo computacional del entrenamiento.

6. Configuración del Entrenamiento

El entrenamiento se realizó utilizando la clase SFTTrainer, con los siguientes parámetros:

```
sft_config = SFTConfig(
    output_dir=output_dir,
    num_train_epochs=1,
    per_device_train_batch_size=1,
    gradient_accumulation_steps=4,
    learning_rate=5e-4,
    logging_steps=10,
    save_steps=200,
    save_total_limit=2,
    fp16=True,
    report_to="none",
    max_length=256,
    dataset_text_field="text"
)
```

Figura 6: Configuración del entrenamiento

Se optó por un número reducido de épocas para evitar el sobreajuste y mantener la capacidad de generalización del modelo.

7. Entrenamiento del Modelo

El entrenamiento se llevó a cabo en **Google Colab**, aprovechando el uso de GPU para acelerar el proceso.

```
trainer = SFTTrainer(
    model=model,
    args=sft_config,
    train_dataset=dataset,
    processing_class=tokenizer,
    peft_config=lora_config
)

# ENTRENAR

trainer.train()
```

Figura 7: Entrenamiento del modelo

8. Almacenamiento del Modelo

Una vez finalizado el entrenamiento, el modelo fue guardado en Google Drive para su posterior uso:

```
trainer.model.save_pretrained(output_dir)
tokenizer.save_pretrained(output_dir)
```

Figura 8. Modelo guardado

9. Pruebas en Entorno Local

El modelo entrenado fue descargado y probado en un entorno local, utilizando la biblioteca peft para cargar el modelo LoRA sobre el modelo base.

```
base_model = AutoModelForCausalLM.from_pretrained(
    base_model_id,
    torch_dtype=dtype,
    device_map=None
)

base_model = base_model.to(device)

model = PeftModel.from_pretrained(
    base_model,
    lora_path,
    is_trainable=False
)
```

Figura 9: Carga del Modelo

Para la generación de respuestas se utilizó un enfoque determinista:

```
with torch.no_grad():
    outputs = model.generate(
        **inputs,
        max_new_tokens=40,
        do_sample=False,
        pad_token_id=tokenizer.eos_token_id,
        eos_token_id=tokenizer.eos_token_id
    )
```

Figura 10. Enfoque de generación

Esto garantiza que el modelo genere siempre el mismo comando para una misma entrada.

Resultados

Una vez finalizado el proceso de entrenamiento del modelo, se realizaron diversas pruebas con el objetivo de evaluar su capacidad para generar comandos de Linux a partir de instrucciones en lenguaje natural. Estas pruebas se llevaron a cabo en un entorno local, utilizando el modelo ajustado mediante LoRA.

Para la evaluación, se definió un conjunto de instrucciones que cubren diferentes categorías de comandos, incluyendo manipulación de archivos, búsqueda, uso de sistema,

red, procesos y compresión. El modelo fue capaz de generar comandos correctos y coherentes en la mayoría de los casos.

A continuación, se presentan algunos ejemplos representativos de los resultados obtenidos:

```
--------------------------------------------------
INPUT: Install a package called git
[transformers] Both `max_new_tokens` (=40) and `max_length`(=131072) seem to have been set. `max_new_tokens` will take precedence. Please refer t
o the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)
OUTPUT: sudo apt install git
--------------------------------------------------
INPUT: Remove package nginx
[transformers] Both `max_new_tokens` (=40) and `max_length`(=131072) seem to have been set. `max_new_tokens` will take precedence. Please refer t
o the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)
OUTPUT: sudo apt remove nginx
--------------------------------------------------
INPUT: Update package list
[transformers] Both `max_new_tokens` (=40) and `max_length`(=131072) seem to have been set. `max_new_tokens` will take precedence. Please refer t
o the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)
OUTPUT: sudo apt update
--------------------------------------------------
INPUT: Start nginx service
[transformers] Both `max_new_tokens` (=40) and `max_length`(=131072) seem to have been set. `max_new_tokens` will take precedence. Please refer t
o the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)
OUTPUT: systemctl start nginx
--------------------------------------------------
INPUT: Stop nginx service
[transformers] Both `max_new_tokens` (=40) and `max_length`(=131072) seem to have been set. `max_new_tokens` will take precedence. Please refer t
o the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)
OUTPUT: systemctl stop service_name
--------------------------------------------------
INPUT: Restart nginx service
[transformers] Both `max_new_tokens` (=40) and `max_length`(=131072) seem to have been set. `max_new_tokens` will take precedence. Please refer t
o the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)
OUTPUT: systemctl restart nginx
--------------------------------------------------
INPUT: Check nginx service status
[transformers] Both `max_new_tokens` (=40) and `max_length`(=131072) seem to have been set. `max_new_tokens` will take precedence. Please refer t
o the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)
OUTPUT: systemctl status nginx
--------------------------------------------------
INPUT: Make a push on git
[transformers] Both `max_new_tokens` (=40) and `max_length`(=131072) seem to have been set. `max_new_tokens` will take precedence. Please refer t
o the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)
OUTPUT: git push
--------------------------------------------------
INPUT: How do I find all files ending in .log in the current folder?
[transformers] Both `max_new_tokens` (=40) and `max_length`(=131072) seem to have been set. `max_new_tokens` will take precedence. Please refer t
o the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)
OUTPUT: find . -type f -name "*.log"
--------------------------------------------------
```

Figura 11. Respuestas del modelo

```
INPUT: Create a directory called backups
[transformers] Both `max_new_tokens` (=40) and `max_length`(=131072) seem to have been set. `max_new_tokens` will take precedence. Please refer t
o the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)
OUTPUT: mkdir backups
--------------------------------------------------
INPUT: Remove an empty directory called old_folder
[transformers] Both `max_new_tokens` (=40) and `max_length`(=131072) seem to have been set. `max_new_tokens` will take precedence. Please refer t
o the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)
OUTPUT: rmdir old_folder
--------------------------------------------------
INPUT: Delete a file called notes.txt
[transformers] Both `max_new_tokens` (=40) and `max_length`(=131072) seem to have been set. `max_new_tokens` will take precedence. Please refer t
o the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)
OUTPUT: rm notes.txt
--------------------------------------------------
INPUT: Copy file.txt to the backup directory
[transformers] Both `max_new_tokens` (=40) and `max_length`(=131072) seem to have been set. `max_new_tokens` will take precedence. Please refer t
o the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)
OUTPUT: cp file.txt /path/to/backup
--------------------------------------------------
INPUT: Move report.pdf to the Documents folder
[transformers] Both `max_new_tokens` (=40) and `max_length`(=131072) seem to have been set. `max_new_tokens` will take precedence. Please refer t
o the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)
OUTPUT: mv report.pdf /path/to/destination
--------------------------------------------------
INPUT: Rename old.txt to new.txt
[transformers] Both `max_new_tokens` (=40) and `max_length`(=131072) seem to have been set. `max_new_tokens` will take precedence. Please refer t
o the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)
OUTPUT: mv old.txt new.txt
--------------------------------------------------
INPUT: Show the content of file.txt
[transformers] Both `max_new_tokens` (=40) and `max_length`(=131072) seem to have been set. `max_new_tokens` will take precedence. Please refer t
o the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)
OUTPUT: cat file.txt
--------------------------------------------------
INPUT: Show the first 5 lines of a file called log.txt
[transformers] Both `max_new_tokens` (=40) and `max_length`(=131072) seem to have been set. `max_new_tokens` will take precedence. Please refer t
o the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)
OUTPUT: head -n 5 log.txt
--------------------------------------------------
INPUT: Show the first 20 lines of notes.txt
[transformers] Both `max_new_tokens` (=40) and `max_length`(=131072) seem to have been set. `max_new_tokens` will take precedence. Please refer t
o the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)
OUTPUT: head -n 20 notes.txt
--------------------------------------------------
```

Figura 12. Respuestas del modelo

Análisis de Resultados

Los resultados obtenidos muestran que el modelo es capaz de interpretar correctamente diferentes formas de expresar una misma intención en lenguaje natural y traducirlas a comandos válidos de Linux.

Se observa que el modelo:

- Genera comandos sintácticamente correctos
- Mantiene coherencia entre la instrucción y la salida
- Responde adecuadamente a distintas variantes lingüísticas
- Produce salidas concisas sin incluir explicaciones adicionales

Además, el uso de generación determinista permitió obtener respuestas consistentes, lo cual es deseable en aplicaciones donde se requiere precisión.

El modelo demuestra una adecuada capacidad de generalización, siendo capaz de responder correctamente a instrucciones que no necesariamente coinciden exactamente con los ejemplos del dataset.

Conclusiones y recomendaciones

En esta práctica se logró desarrollar un modelo de lenguaje capaz de generar comandos de Linux a partir de instrucciones en lenguaje natural, utilizando técnicas modernas de procesamiento de lenguaje natural basadas en la arquitectura Transformer.

El uso de un modelo preentrenado de la familia LLaMA permitió aprovechar conocimiento previo del lenguaje, facilitando su adaptación a la tarea específica mediante fine-tuning. Asimismo, la implementación de la técnica LoRA (Low-Rank Adaptation) permitió realizar este ajuste de manera eficiente, reduciendo significativamente el número de parámetros a entrenar y optimizando el uso de recursos computacionales.

El modelo mostró un comportamiento consistente en la generación de comandos, logrando interpretar correctamente diferentes formas de expresar una misma intención. Esto indica que el sistema no solo memoriza ejemplos, sino que también es capaz de generalizar a nuevas entradas.

Además, el uso de un formato estructurado tipo instrucción-respuesta permitió guiar adecuadamente al modelo, asegurando que las salidas fueran concisas y sin información adicional.

Experimentos Realizados

Durante el desarrollo de la práctica se llevaron a cabo diversos experimentos con el objetivo de mejorar el desempeño del modelo y comprender su comportamiento.

1. Variación del Dataset

Se realizaron pruebas incrementando el tamaño del dataset, agregando múltiples formas de expresar una misma instrucción. Esto permitió mejorar la capacidad de generalización del modelo, ya que fue expuesto a distintas estructuras lingüísticas.

También se realizaron ajustes en la calidad del dataset, priorizando ejemplos claros, consistentes y representativos del dominio de comandos de Linux.

2. Ajuste de Parámetros de Entrenamiento

Se experimentó con diferentes configuraciones de entrenamiento, incluyendo:

- Número de épocas (epochs)
- Tasa de aprendizaje (learning rate)
- Tamaño del batch

Se observó que un número reducido de épocas (1 epoch) combinado con una tasa de aprendizaje moderada permitió obtener mejores resultados, evitando el sobreajuste del modelo.

3. Configuración de LoRA

Se probaron diferentes configuraciones de LoRA, ajustando parámetros como:

- Rango (r)
- Alpha
- Módulos objetivo

Finalmente, se seleccionó una configuración ligera que permitió adaptar el modelo sin afectar su estabilidad, enfocando el aprendizaje en componentes clave del Transformer.

4. Estrategias de Generación

Se evaluaron distintos métodos de generación de texto. Se optó por un enfoque determinista utilizando:

do_sample = False

Esta configuración permitió obtener respuestas consistentes y precisas, lo cual es fundamental para una tarea donde se requiere exactitud en los comandos generados.

5. Evaluación en Entorno Local

Después del entrenamiento en Google Colab, el modelo fue probado en un entorno local. Esto permitió verificar su portabilidad y funcionamiento fuera del entorno de entrenamiento.

Las pruebas incluyeron múltiples categorías de comandos, validando que el modelo responde adecuadamente en distintos escenarios.

Observaciones

Durante el desarrollo de la práctica se identificaron aspectos importantes relacionados con el comportamiento del modelo:

- La calidad del dataset tiene un impacto directo en el desempeño del modelo
- El uso de múltiples variantes lingüísticas mejora significativamente la generalización
- Configuraciones de entrenamiento más largas no necesariamente producen mejores resultados
- El uso de LoRA permite entrenar modelos grandes con recursos limitados
- La generación determinista es adecuada para tareas estructuradas

Trabajo Futuro

Como trabajo futuro, se proponen las siguientes mejoras:

- Ampliar el dataset con comandos más avanzados y específicos
- Incorporar soporte para parámetros dinámicos en los comandos
- Implementar un sistema interactivo que permita múltiples turnos de conversación
- Integrar el modelo en una aplicación o API para su uso práctico
- Evaluar el modelo utilizando métricas cuantitativas adicionales

Bibliografía

- Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019).
  BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.
  Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL).
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017).
  Attention Is All You Need.
  Advances in Neural Information Processing Systems (NeurIPS).
- Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M. A., Lacroix, T., Rozière, B., et al. (2023).
  LLaMA: Open and Efficient Foundation Language Models.
  Meta AI.
- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, L., & Chen, W. (2021).
  LoRA: Low-Rank Adaptation of Large Language Models.
  arXiv preprint arXiv:2106.09685.
- Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., et al. (2020).
  Transformers: State-of-the-Art Natural Language Processing.
  Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP).
- Hugging Face. (2026).
  Transformers Documentation.
  Disponible en: https://huggingface.co/docs/transformers
- Hugging Face. (2026).
  Datasets Documentation.
  Disponible en: https://huggingface.co/docs/datasets

- Hugging Face. (2026).
  PEFT (Parameter-Efficient Fine-Tuning) Documentation.
  Disponible en: https://huggingface.co/docs/peft
- PyTorch Team. (2026).
  PyTorch Documentation.
  Disponible en: https://pytorch.org/docs
- Google. (2026).
  Google Colab Documentation.
  Disponible en: https://colab.research.google.com/